

Daily Problem: 16–Feb–2026

Problem Statement

Similar to a push-down automaton (PDA) with a stack, consider a finite-state automaton equipped with a single unbounded FIFO queue (a *queue automaton*). Show that any language accepted by a PDA (with a stack) is also accepted by a finite-state automaton with a queue.

Definitions

A PDA has a finite control and a stack supporting:

- push: add symbols to the top,
- pop: remove the top symbol,
- top-test: inspect the top symbol (or $\perp$ if empty).

A queue automaton has a finite control and a FIFO queue supporting:

- enqueue: add a symbol to the *rear*,
- dequeue: remove a symbol from the *front*,
- front-test: inspect the front symbol (or $\perp$ if empty).

We assume both machines read the input left-to-right and may use ε -moves.

Main Idea

We show how to simulate the PDA's stack using the queue. The difficulty is that the stack is LIFO while the queue is FIFO. The standard trick is to store the entire stack in the queue and use ε -moves to *rotate* the queue so that the symbol we want to treat as “top of stack” comes to the front.

Claim

For every PDA P , there exists a queue automaton Q such that

$$L(Q) = L(P).$$

Proof

Let

$$P = (Q_P, \Sigma, \Gamma, \delta_P, q_0, Z_0, F_P)$$

be a PDA, where Γ is the stack alphabet, $Z_0 \in \Gamma$ is the initial stack symbol, and acceptance is by final state (acceptance by empty stack can be reduced to final-state acceptance in the usual way).

We build a queue automaton

$$Q = (Q_Q, \Sigma, \Gamma', \delta_Q, q'_0, F_Q)$$

that simulates P . The queue alphabet Γ' will contain:

- the stack symbols Γ ,
- a special marker $\# \notin \Gamma$.

Queue invariant

At all times, the queue content has the form

$$\# s_1 s_2 \cdots s_k,$$

where $s_1 \cdots s_k \in \Gamma^*$ encodes the current stack content *from bottom to top*:

$$\text{stack bottom} = s_1, \dots, \text{stack top} = s_k.$$

Thus the *top of the simulated stack* is the *last* symbol of the queue (at the rear), not at the front. We will access it by rotating.

Initially, the PDA stack is just Z_0 . We initialize the queue as

$$\#Z_0.$$

Queue rotation subroutine

A key operation we need is: bring the rear symbol (the simulated top-of-stack) to the front, while preserving the cyclic order.

Because the queue is FIFO, we can rotate by repeatedly:

dequeue front symbol x and enqueue it at rear.

If we do this enough times, a desired position will appear at the front.

To know when one full rotation has happened, we use the marker $\#$. The automaton can rotate until it sees $\#$ again at the front. This allows us to “scan” the entire queue using only dequeue+enqueue and finite control.

Simulating a PDA transition

A PDA transition has the form:

$$\delta_P(q, a, X) \ni (q', \alpha)$$

meaning: in state q , reading input symbol $a \in \Sigma \cup \{\varepsilon\}$, if the top of stack is $X \in \Gamma$, then move to state q' and replace X by the string $\alpha \in \Gamma^*$ (push α after popping X).

We now show how Q performs the same effect on the queue encoding.

Step 1: Bring the simulated top symbol to the front.

Given the invariant $\#s_1 \cdots s_k$, the top is s_k at the rear. The automaton rotates the queue until $\#$ is at the front. Then it performs one more dequeue+enqueue step repeatedly to traverse the queue. Concretely, it rotates the queue while counting in the finite control whether it has just passed $\#$. After one full pass, it can position the queue so that the last symbol s_k becomes the next dequeued symbol. (Informally: by rotating through the entire queue, we can bring any position to the front.)

Because Q may use ε -moves, it can perform this rotation without consuming input.

Step 2: Check and remove the top symbol.

Once s_k is at the front, Q checks whether it equals X . If not, this simulation branch rejects. If yes, it dequeues X , thereby popping the simulated top of stack.

Now the queue contains $\#s_1 \cdots s_{k-1}$ (up to rotation).

Step 3: Push α onto the simulated stack.

To push $\alpha = Y_1 Y_2 \cdots Y_t$ onto the stack, we must append these symbols to the *rear* of the queue encoding. That is exactly what enqueue does. So Q enqueues $Y_1, \dots, Y_t$ (in that order) at the rear.

After this, the queue encodes $\#s_1 \cdots s_{k-1} Y_1 \cdots Y_t$, which corresponds to replacing top X by α , exactly like the PDA.

Step 4: Consume the same input symbol.

If the PDA transition reads $a \in \Sigma$, then Q also reads a . If it is an ε -move, Q consumes no input.

Finally, Q updates its finite control state from q to q' .

Correctness

We show that Q simulates P step-for-step.

Simulation invariant. After reading the same input prefix, the control state of Q equals the control state of P , and the queue content of Q encodes the stack content of P (bottom-to-top), up to cyclic rotation.

Preservation. Given the invariant, the above procedure:

- correctly identifies and removes the top-of-stack symbol,
- correctly appends the pushed string α to the rear,
- and updates the control state and input head in the same way as P .

Hence the invariant is preserved after each simulated PDA transition.

Acceptance. If P accepts by reaching a final state after consuming all input, then Q reaches the same final state after consuming all input, and we set $F_Q = F_P$ (appropriately lifted to the state space of Q). Conversely, any accepting run of Q corresponds to a valid sequence of simulated PDA moves, hence P accepts.

Therefore, $L(Q) = L(P)$.

Conclusion

For any PDA P , we can construct a finite-state automaton with a queue Q that simulates P and accepts the same language. Hence every language accepted by a push-down automaton (stack) is also accepted by a finite-state automaton with a queue.